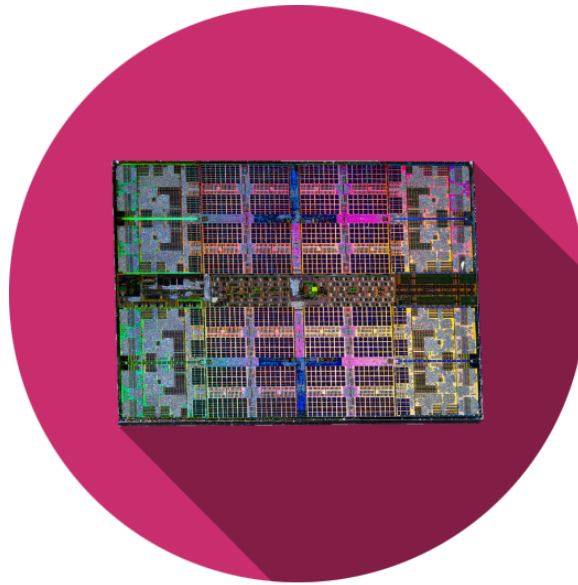




Computer Architecture

CAR - PROJET - HEIRV

Gabriel Zeizer, Gonin Raphael

14 juin 2026

Table des matières

1 – Introduction	3
2 – Objectif du laboratoire	3
3 – Spécification	3
4 – Matériel utilisé	4
4.1 – Motherboard (EBS3)	4
4.2 – Daughterboard (LFE5U-25F)	4
4.3 – Carte boutons et LEDs	5
4.4 – Boards supplémentaires	5
5 – Outils	6
5.1 – HDL Designer	6
5.2 – ModelSim	6
5.3 – Lattice Diamond	6
6 – Design	6
6.1 – Blocs fournis	6
7 – Bases Théoriques	6
8 – Circuit	8
8.1 – Top Level	8
8.2 – Unité de contrôle	8
8.3 – Signaux de contrôle	10
8.4 – Opcodes des types d'instructions	10
8.5 – Table de vérité de la FSM	11
8.6 – Table de décodage ALU	11
8.7 – Table de décodage immédiat (Instr. Decoder)	12
8.8 – Justification des choix d'implémentation	12
9 – Simulation	13
10 – Implémentation	13
10.1 – Circuit HEIRV32	13
10.2 – Code assembleur personnalisé	13
11 – Conclusion	17
11.1 – Difficultés rencontrées	17
11.2 – Etapes suivantes	17

1 – Introduction

Ce projet s'inscrit dans le cadre du cours d'Architecture des ordinateurs de la filière Informatique et Systèmes de Communication à la HES-SO Valais/Wallis. L'objectif est de réaliser un processeur RISC-V 32 bits, simulé sous ModelSim puis déployé physiquement sur une carte FPGA EBS 3.

Le travail couvre trois points principaux : la conception du chemin de données en VHDL, l'implémentation de l'unité de contrôle (FSM, décodeur ALU, décodeur d'instruction), et l'écriture d'un programme en assembleur HEIRV32 exploitant les boutons et les LEDs de la carte d'extension.

2 – Objectif du laboratoire

L'objectif de ce projet est de développer une architecture de processeur simple, de type RISC-V. Nous allons essentiellement nous concentrer sur la conception de l'unité de contrôle. Nous allons également implémenter un ensemble d'instructions de base pour permettre au processeur d'exécuter des programmes simples.

La partie matérielle, comprenant une carte de développement FPGA ainsi que plusieurs boutons et LEDs pour interagir avec le processeur, nous est fournie.

La deuxième partie du projet consistera à créer un programme en assembleur qui sera exécuté sur notre processeur, et qui permettra de tester les différentes fonctionnalités que nous avons implémentées.

3 – Spécification

Le processeur que nous allons implémenter doit être capable d'exécuter les instructions de base suivantes:

- Type R : **add, sub, and, or, slt, xor, sll, srl**
- Type I : **addi, andi, ori, slti, xori, slli, srli**
- Type mémoire : **lw, sw**
- Type saut : **beq, jal, jalr**

Il doit également pouvoir écouter l'activation d'un bouton pour déclencher l'exécution du programme, et afficher le résultat de l'exécution sur les LEDs.

4 – Matériel utilisé

Le système EBS3 repose sur trois platines : une motherboard, une daughterboard portant le FPGA, et une carte boutons/LEDs.

4.1 – Motherboard (EBS3)

La carte mère (Figure 1) expose les interfaces vers le monde extérieur :

- 4 connecteurs **PMod**
- 2 ports parallèles
- Port **Gigabit Ethernet** (contrôleur VSC8531, 125Mhz)
- Port **USB-C** (contrôleur CP2102N)
- Bouton **Hard Reset** (signal `nRESET_IN`)
- Connecteur JTAG extender

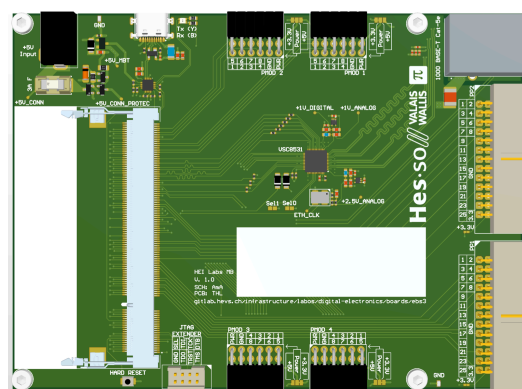


Figure 1 - Carte Mère

4.2 – Daughterboard (LFE5U-25F)

La carte fille (Figure 2) héberge le FPGA et ses périphériques directs. Elle se connecte à la motherboard via un connecteur **SODIMM-200**.

Composant	Détail
FPGA	Lattice LFE5U-25F (famille ECP5)
Horloge	100 MHz
Programmes	QSPI Flash - AT25SF321B
DRAM	256 Mb synchrone - IS42S16160J
Stockage code	Slot micro-SD
Flashing	USB/JTAG
Alimentation	USB-C +5V ou via motherboard
Tensions internes	+3.3 V / +2.5 V / +1.1 V

Trois LEDs indiquent l'état du système :

- **LED bicolore (bleu/jaune)** : bleu signifie que la board est en train de recevoir des données, jaune que des données sont transmises de la board à l'hôte
- **LED verte (LD2)** : programme en cours de chargement depuis la SD (séquence de démarrage active)
- **LED rouge (LD3)** : FPGA sous tension et prêt à être configuré (programme non encore chargé)

Deux boutons sont disponibles sur la daughterboard :

- **Hard Reset** : reset matériel via le circuit APX811-31U (power-on-reset)
- **Soft Reset** : force le rechargement du programme depuis la flash QSPI

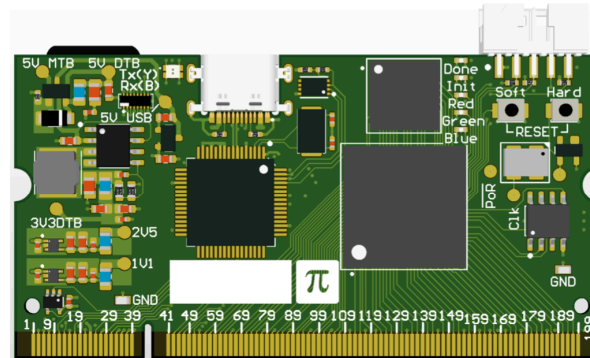


Figure 2 - Carte Fille

4.3 – Carte boutons et LEDs

Cette carte fille secondaire (Figure 3) est connectée à la motherboard. Elle possède **4 boutons** et **8 LEDs**, reliés directement aux registres du processeur RISC-V :

- x30 : registre d'écriture des LEDs (bit 0 = LED 0, bit 1 = LED 1, ...)
- x31 : registre de lecture des boutons (lecture seule ; bit 0 = S1, bit 1 = S2, ...)

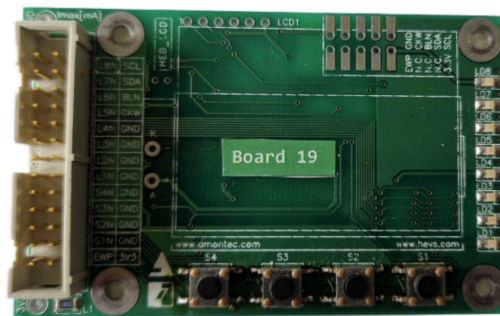


Figure 3 - Carte boutons et LEDs

4.4 – Boards supplémentaires

En plus de la carte de base, plusieurs modules d'extension sont disponibles pour ajouter des fonctionnalités supplémentaires au projet. Deux modules équipés de 8 LEDs chacun (Figure 4) étaient également à notre disposition, mais nous avons choisi de ne pas les utiliser.

Type	Modules
Entrées	PMod BTN, PMod CON1, PMod CON3
Sorties	PMod OD1, PMod BB
E/S	PMod MAXSONAR (capteur à ultrasons)
LEDs	PMod 8LD (8 LEDs supplémentaires)



Figure 4 - Carte supplémentaire LEDs

5 – Outils

5.1 – HDL Designer

HDL Designer est un outil est un outil de conception de circuits numériques. Il s'agit de l'outil que nous avons utilisé tout au long de ce projet afin de concevoir notre implémentation du HEIRV32.

5.2 – ModelSim

ModelSim est un simulateur qui permet de voir les détails du fonctionnement d'un circuit fait avec HDL Designer. Il nous permet de contrôler le bon fonctionnement de notre implémentation à l'aide de tests unitaires ainsi que de manière visuelle grâce à la représentation visuelle des signaux.

5.3 – Lattice Diamond

Lattice Diamond nous permet de flasher la carte FPGA avec un circuit réalisé dans HDL Designer

6 – Design

6.1 – Blocs fournis

Les différents blocs nécessaires à la construction d'un processeur RISC-V ont été fournis et sont listés ci-dessous.

1. HEIRV32_MC
 - **controlUnit** : bloc pour le décodage des instructions
 - **heirv32_mc** : top-level
 - **instructionDataManager** : mémoire du programme groupant instructions et data, capable de lecture et écriture
2. HEIRV32
 - **ALU** : une version de l'ALU capable d'addition, soustraction, AND, OR, et SLT
 - **buffer(Enable)** : buffer clockés (bascules) avec ou sans entrée enable
 - **extend** : bloc d'extension de l'instruction pour les tests, supportant les instructions I, S, B et J
 - **mux3To1ULogVec** : mux 3 vers 1 de std_ulogic_vector
 - **registerFile** : bloc de gestion des 32 registres, remplaçant x31 par le vecteur btns - registre de lecture des boutons - et x30 par le vecteur LEDs - registre d'écriture des LEDs -

7 – Bases Théoriques

Afin de comprendre le fonctionnement du processeur HEIRV32, il est nécessaire d'analyser les caractéristiques de chaque instruction supportée. Dans l'architecture multi-cycle choisie, chaque instruction est décomposée en étapes distinctes, séparées par un coup d'horloge. Le pipeline se construit comme suit :

1. **Fetch** : récupération de l'instruction en mémoire
2. **Decode** : décodage de l'instruction et lecture des registres sources
3. **Execute** : exécution de l'opération dans l'ALU
4. **Memory Access** : accès mémoire (uniquement pour lw)
5. **Write Back** : enregistrement du résultat dans le registre de destination (sauf jal)

Toutes les instructions ne nécessitent pas l'exécution de chaque étape, ces dernières peuvent être dissociées, séparées par un coup d'horloge. Les instructions les plus courtes ne nécessitant que 3 à 4 étapes bénéficieront d'un traitement plus rapide.

7.0.1 – Fetch

Le **fetch** correspond à la récupération de l'instruction depuis la mémoire. Le bloc responsable est l'`instructionDataManagerSDCard`. Il accède à la mémoire grâce au **Program Counter (PC)**, une variable maintenant la position courante dans la mémoire du programme.

L'instruction récupérée est sous forme binaire sur 32 bits, comme démontré dans la Figure 5 :

Address	Instruction	Type	Fields	rs1	f3	rd	op	Machine Lang
0x1000	L7: lw x6, -4(x9)	I	imm[11:0] 111111111100	01001	010	00110	0000011	FFC4A303

Figure 5 - Code binaire d'une instruction

En parallèle du chargement de l'instruction, un ALU séparé est utilisé pendant le Fetch pour calculer l'adresse suivante $PC + 4$.

7.0.2 – Decode

Le **decode** extrait les champs significatifs de la chaîne binaire et génère les signaux de sélection pour l'ensemble des blocs du circuit. Cette étape est gérée par l'**unité de contrôle** (controlUnit).

Les champs extraits sont :

- **opcode bits [6:0]** : identifie le type d'instruction (R, I, S, B, J)
- **funct3 bits [14:12]** : affine le décodage pour les instructions de même type
- **funct7 bits [31:25]** : distingue par exemple add de sub (type R uniquement)
- **rs1, rs2** : adresses des registres sources (bits [19:15] et [24:20])
- **rd** : adresse du registre de destination (bits [11:7])
- **imm** : valeur immédiate, dont la position varie selon le type d'instruction

La répartition de ces champs par type d'instruction est définie dans la Figure 6 :

HEIRV32 Instruction Set

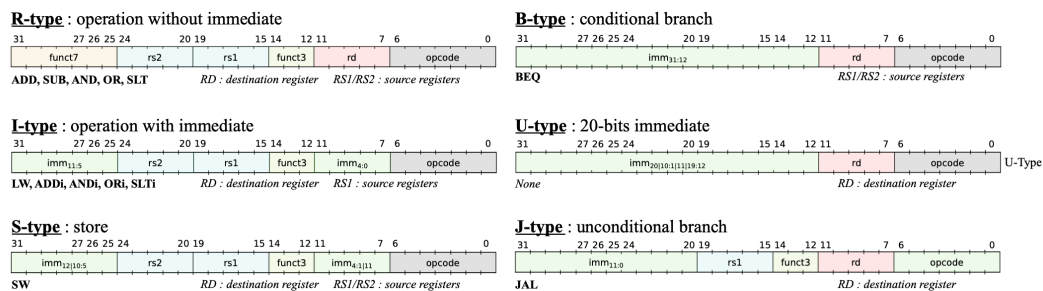


Figure 6 - HEIRV32 Instruction Set

Grâce à cette identification, l'unité de contrôle génère les signaux appropriés pour chaque bloc du circuit, détaillés dans la section suivante.

7.0.3 – Execute

L'étape **execute** lance l'opération dans l'ALU selon les ordres fournis par l'unité de contrôle. Les sources des opérandes A et B de l'ALU sont sélectionnées par les multiplexeurs contrôlés par ALUSrcA et ALUSrcB.

Pour les instructions de type R et I, l'ALU effectue directement l'opération arithmétique ou logique. Pour beq, elle calcule la différence $rs1 - rs2$ afin de déterminer si les deux registres sont égaux via le flag **Zero**. Pour lw et sw, elle calcule l'adresse mémoire $rs1 + imm$.

7.0.4 – Memory Access

L'étape **memory access** ne s'applique, dans cette implémentation, qu'à l'instruction lw. Elle utilise l'adresse calculée à l'étape précédente pour lire la valeur stockée en mémoire.

7.0.5 – Write Back

L'étape **write back** enregistre la valeur résultante dans le registre de destination rd. La source du résultat est déterminée par le signal resultSrc :

- "00" : résultat de l'ALU (instructions R, I)

- "01" : donnée lue en mémoire (instruction lw)
- "10" : résultat ALU bypassed (calcul $PC + 4$)

L'instruction jal ne passe pas par cette étape de la même manière : elle enregistre directement $PC + 4$ comme adresse de retour dans rd, puis met à jour le PC avec l'adresse de saut calculée en **Decode**.

8 – Circuit

8.1 – Top Level

La première étape du projet consiste à relier les différents composants du processeur entre eux (Figure 7). Les différents signaux de contrôle provenant de l'unité de contrôle doivent être correctement acheminés vers les différentes unités fonctionnelles du processeur, telles que l'ALU, les registres, la mémoire, etc.

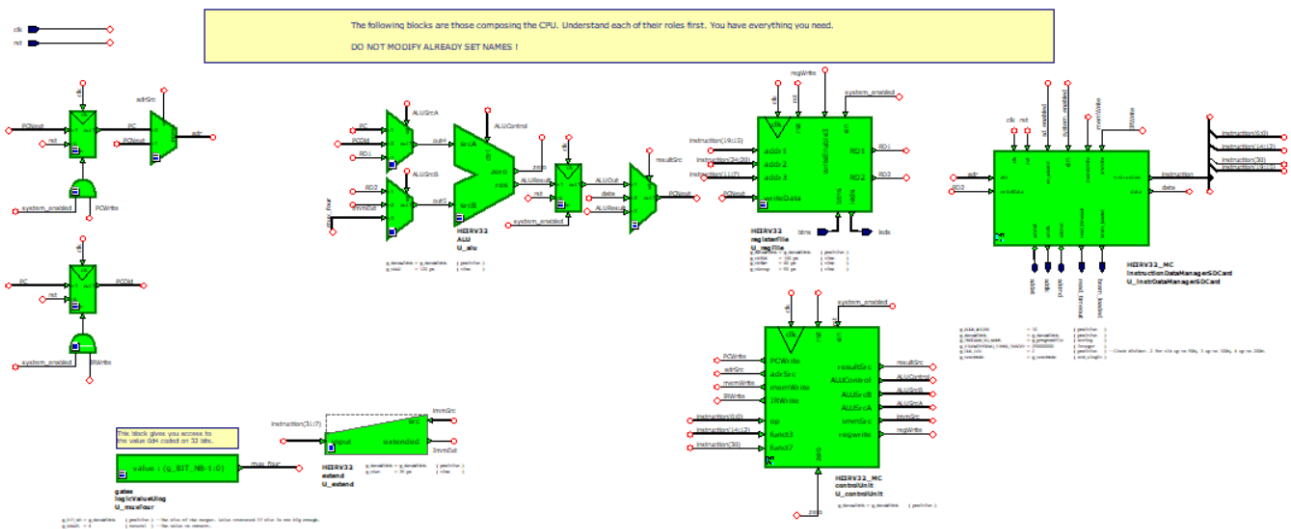


Figure 7 - Vue top-level du système HEIRV32 sur FPGA

8.2 – Unité de contrôle

L'unité de contrôle, présentée dans la Figure 8, est le cœur du processeur, elle reçoit les instructions du programme et produit les signaux contrôlant l'état de tous les éléments du processeur.

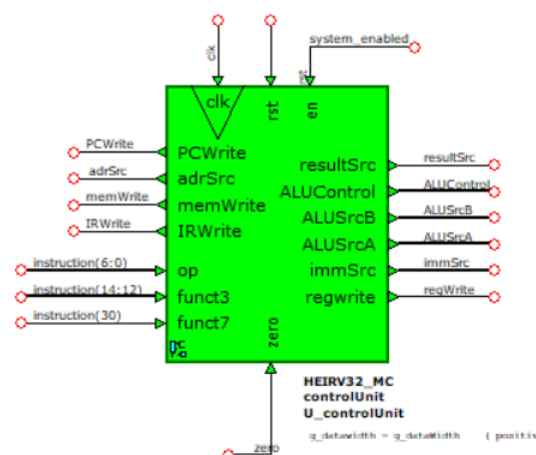


Figure 8 - Vue top-level du Control Unit

8.2.1 – AluDecoder

Le bloc de la Figure 9 est chargé de décoder les instructions découlant du signal **AluOp** et de générer les signaux de contrôle de l'ALU. Il prend également en compte les signaux **func3** et **func7**, comme démontré dans le Listing 1.

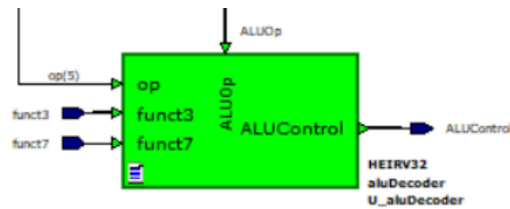


Figure 9 - AluDecoder, situé dans l'unité de contrôle

```

1  -- Setup ALU operation based on instruction
2  ARCHITECTURE studentVersion OF aluDecoder IS
3  BEGIN
4
5  decode : process(op, funct3, funct7, ALUOp)
6  begin
7  case ALUOp is
8  when "00" => ALUControl <= "000"; -- Operation ADD
9  when "01" => ALUControl <= "001"; -- Operation SUB
10 when others =>
11   if (funct3 = "000" and op = '1' and funct7 = '1') then
12     ALUControl <= "001"; -- Operation SUB
13   elsif (funct3 = "000") then
14     ALUControl <= "000"; -- Operation ADD
15   elsif (funct3 = "101" and funct7 = '0') then
16     ALUControl <= "111"; -- Operation SRL
17   elsif (funct3 = "001") then
18     ALUControl <= "110"; -- Operation SLL
19   elsif (funct3 = "010") then
20     ALUControl <= "101"; -- Operation SLT
21   elsif (funct3 = "100") then
22     ALUControl <= "100"; -- Operation XOR
23   elsif (funct3 = "110") then
24     ALUControl <= "011"; -- Operation OR
25   elsif (funct3 = "111") then
26     ALUControl <= "010"; -- Operation AND
27   else
28     ALUControl <= "000"; -- Default operation ADD
29   end if;
30 end case;
31 end process decode;
32
33 END ARCHITECTURE studentVersion;

```

Listing 1 - Code VHDL AluDecoder

8.2.2 – InstrDecoder

Ce bloc, décrit par le Listing 2, s'occupe de décoder les instructions provenant du signal **Op** et de produire le signal de contrôle pour le bloc **extend**.

```

1  -- Defines extend type based on instruction
2  ARCHITECTURE studentVersion OF instrDecoder IS
3  BEGIN
4
5  decode : process(op)
6  begin
7  case op is
8  when "0010011" => immSrc <= "00"; -- I-Type
9  when "0100011" => immSrc <= "01"; -- S-Type
10 when "1100011" => immSrc <= "10"; -- B-Type
11 when "1101111" => immSrc <= "11"; -- J-Type
12 when "1100111" => immSrc <= "00"; -- J-Type (Jahr)
13 when "0000011" => immSrc <= "00"; -- Lw-Type
14 when others => immSrc <= "--"; -- Others
15 end case;
16 end process decode;
17

```

Listing 2 - Code VHDL InstrDecoder

8.2.3 – Main FSM

La **Main FSM** (Figure 10) est le cœur de l'unité de contrôle. Elle orchestre les étapes du pipeline multi-cycle en générant les signaux de contrôle adéquats à chaque état.

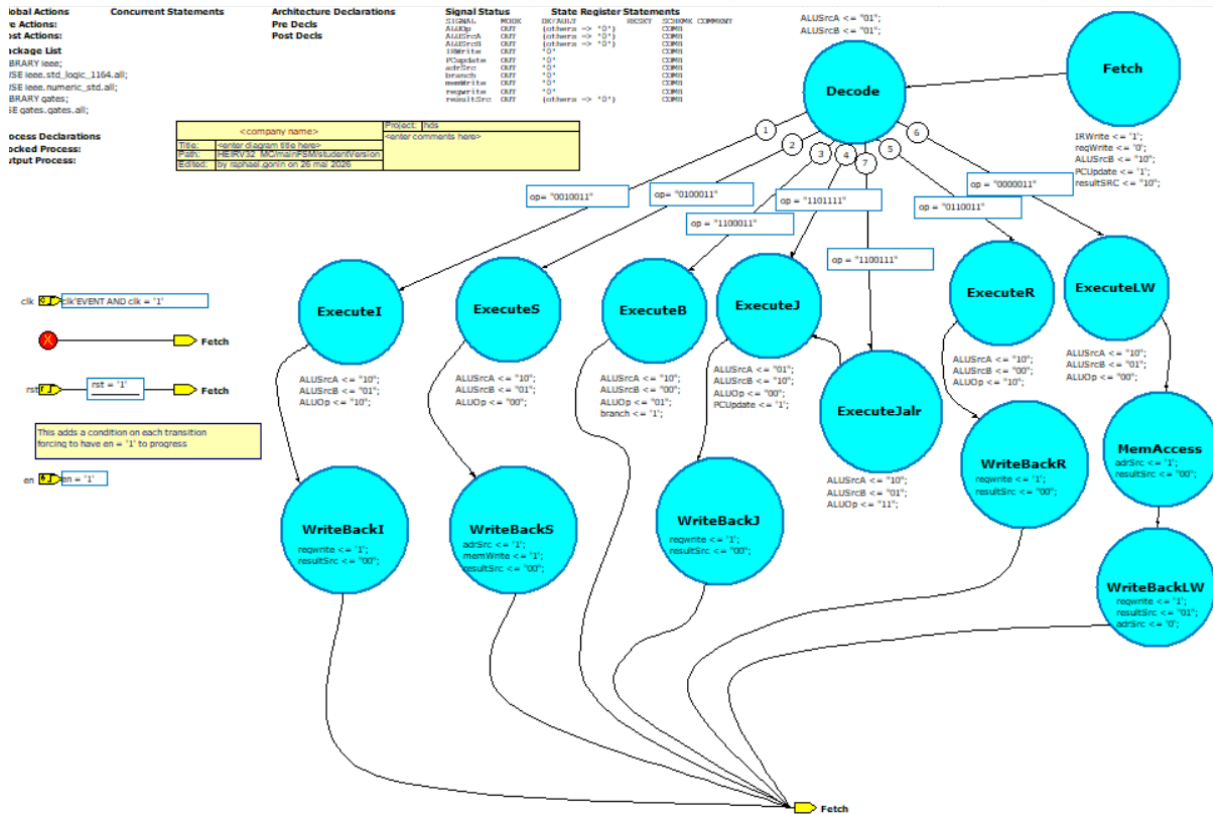


Figure 10 - Main FSM

8.3 – Signaux de contrôle

La FSM génère les signaux détaillés dans la Table 1 (valeur par défaut = '0' ou "00") :

Signal	Taille	Rôle
IRWrite	1 bit	Autorise l'écriture du registre instruction
PCUpdate	1 bit	Mise à jour inconditionnelle du PC
branch	1 bit	Activation du branchement conditionnel (BEQ)
AdrSrc	1 bit	Sélection source adresse mémoire (0=PC, 1=ALU)
MemWrite	1 bit	Écriture mémoire
regWrite	1 bit	Écriture dans le fichier de registres
ALUSrcA	2 bits	Source A de l'ALU (00=PC, 01=oldPC, 10=RS1)
ALUSrcB	2 bits	Source B de l'ALU (00=RS2, 01=imm, 10=4)
ALUOp	2 bits	Mode ALU (00=add, 01=sub, 10=instruction)
resultSrc	2 bits	Source du résultat (00=ALU, 01=mémoire, 10=oldALU)

Table 1 - Signaux de contrôle générés par la FSM

8.4 – Opcodes des types d'instructions

Les transitions depuis Decode sont déterminées par op[6:0] dans la Table 2::

Type	op[6:0]	Instructions
R	0110011	add, sub, and, or, slt, xor, sll, srl
I	0010011	addi, andi, ori, slti, xori, slli, srli
LW	0000011	lw
SW	0100011	sw
BEQ	1100011	beq
JAL	1101111	jal
JALR	1100111	jalr

Table 2 - Opcodes des types d'instructions

8.5 – Table de vérité de la FSM

Chaque ligne de la Table 3 correspond à un état et ses sorties associées. Les signaux non listés sont à leur valeur par défaut ('0' / "00").

État	IRW	PCUpd	br	AdrSrc	MemW	regW	ALUSrcA	ALUSrcB	ALUOp	resultSrc
Fetch	1	1	0	0	0	0	00	10	00	10
Decode	0	0	0	0	0	0	01	01	00	--
ExecuteI	0	0	0	0	0	0	10	01	10	--
ExecuteR	0	0	0	0	0	0	10	00	10	--
ExecuteLW	0	0	0	0	0	0	10	01	00	--
ExecuteS	0	0	0	0	0	0	10	01	00	--
ExecuteB	0	0	1	0	0	0	10	00	01	--
ExecuteJ	0	1	0	0	0	0	01	10	00	--
ExecuteJalr	0	0	0	0	0	0	10	01	11	--
WriteBackI	0	0	0	0	0	1	--	--	--	00
WriteBackR	0	0	0	0	0	1	--	--	--	00
MemAccess	0	0	0	1	0	0	--	--	--	00
WriteBackLW	0	0	0	0	0	1	--	--	--	01
WriteBackS	0	0	0	1	1	0	--	--	--	00
WriteBackJ	0	0	0	0	0	1	--	--	--	00

Table 3 - Table de vérité de la FSM

Légende : IRW =IRWrite, PCUpd =PCUpdate, br =branch, MemW =MemWrite, regW =regWrite.

8.6 – Table de décodage ALU

La Table 4 correspond au code que l'on retrouve au Chapitre 8.2.1 :

ALUOp	funct3	Op5·funct7[5]	Instruction	ALUControl
00	---	--	lw, sw	000 (add)
01	---	--	beq	001 (sub)
10	000	0x, 10	add / addi	000 (add)
10	000	11	sub	001 (sub)
10	001	--	sll / slli	110 (sll)
10	010	--	slt / slti	101 (slt)
10	100	--	xor / xori	100 (xor)
10	101	x0	srl / srli	111 (srl)
10	110	--	or / ori	011 (or)
10	111	--	and / andi	010 (and)
11	---	--	jalr (addr step)	000 (add)

Table 4 - Table de décodage de l'ALU

8.7 – Table de décodage immédiat (Instr. Decoder)

L'opcode indique le format de l'immédiat via le signal `immSrc` au bloc d'extension immédiate, afin que ce dernier puisse le reconstituer correctement avant de l'étendre à 32 bits.

op[6:0]	immSrc	Type
0010011 / 0000011 / 1100111	00	I
0100011	01	S
1100011	10	B
1101111	11	J

Table 5 - Table de décodage du signal envoyé à l'extension immédiate

8.8 – Justification des choix d'implémentation

8.8.1 – JALR : deux étapes d'exécution

`jalr` saute à `rs1 + imm`, contrairement à `jal` qui saute à `oldPC + imm`. Pendant le décodage, `rs1` n'est pas encore disponible en sortie du fichier de registres (la bascule interne le retarde d'un cycle). Il est donc impossible de calculer l'adresse de saut en **Decode** comme pour `jal`. La solution adoptée est l'ajout d'un état intermédiaire **ExecuteJalr** qui effectue le calcul `rs1 + imm`, puis **ExecuteJ** qui met à jour le PC et calcule `oldPC + 4`. `ALUOp = "11"` est utilisé pour forcer l'addition dans cet état sans passer par le décodage instruction.

8.8.2 – Signal branch vs PCUpdate

La mise à jour du PC est séparée en deux chemins :

- `PCUpdate` : mise à jour inconditionnelle (utilisée en Fetch pour PC+4, et en ExecuteJ).
- `branch AND Zero` : mise à jour conditionnelle (BEQ uniquement, si les deux registres sont égaux).

Ces deux signaux sont combinés via un OR : `PCWrite = PCUpdate OR (branch AND Zero)`. Cela évite d'avoir à gérer le flag `Zero` dans tous les états.

8.8.3 – ALUSrcA = "01" en Decode

Pendant le décodage, l'ALU est utilisé pour pré-calculer l'adresse de saut `oldPC + imm` (utile pour BEQ et JAL). La source A est donc `oldPC` (valeur du PC avant l'incrément), maintenu dans une bascule dédiée mise à jour à chaque Fetch.

9 – Simulation

La simulation avec ModelSim a permis de corriger plusieurs erreurs, notamment sur les valeurs générées par l'unité de contrôle. En observant les états internes et les résultats des tests unitaires, nous avons pu identifier des défauts d'implémentation dans la machine d'états (FSM) : certaines transitions étaient mal conditionnées ou les signaux de sortie n'étaient pas mis à jour au bon moment. Ces anomalies ont pu être localisées puis corrigées.

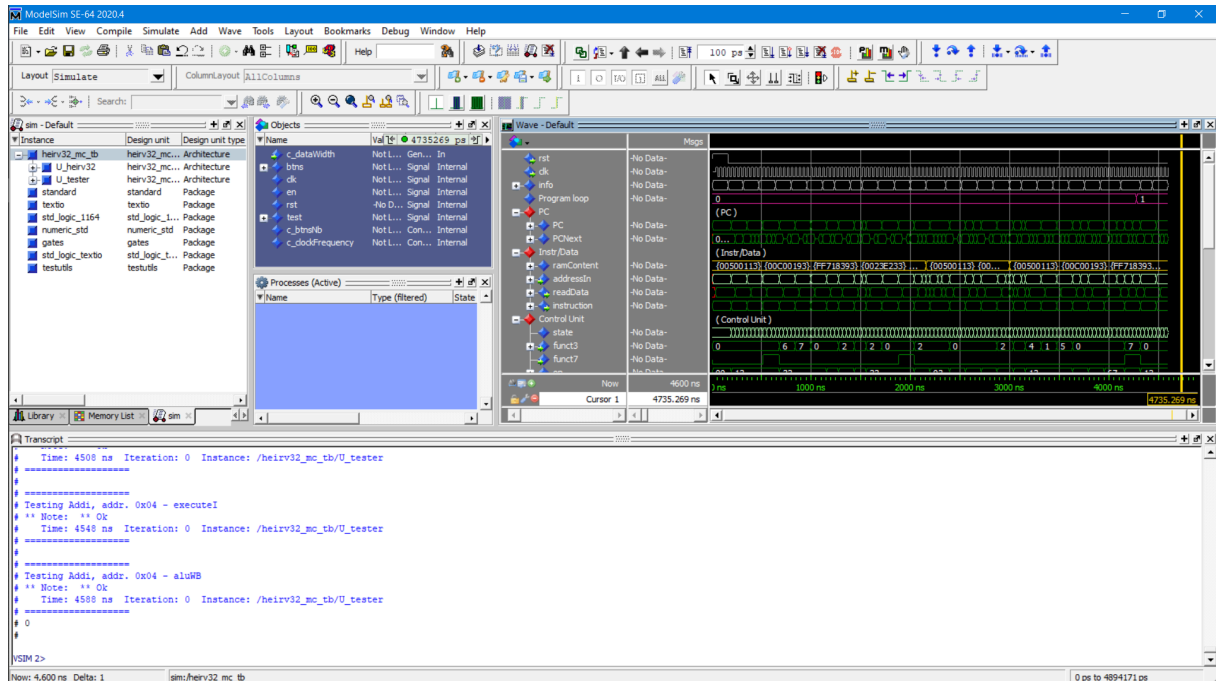


Figure 11 - Simulation avec ModelSim

10 – Implémentation

Une fois le fonctionnement validé en simulation, le circuit et le code sont transférés sur la carte de développement FPGA.

10.1 – Circuit HEIRV32

À l'aide de l'outil Lattice Diamond, le circuit est compilé, prêt à être chargé sur la FPGA.

Le circuit est d'abord testé avec le code préalablement chargé. Puis, après avoir vérifié le bon fonctionnement du circuit, nous pouvons passer à la création de notre propre code assembleur.

10.2 – Code assembleur personnalisé

Le code assembleur (Listing 3) est chargé directement sur la carte micro-SD après avoir été compilé par le programme : HEIRV32-ASM_1.2.5. Ce dernier retourne un fichier .bin, prêt à être copié sur la carte, à partir de notre fichier assembleur .c

Nous avons décidé de faire un code permettant d'afficher une chenille avec les LEDs. Les fonctions sont les suivantes:

- Une pression sur le bouton S1 démarre la chenille
- Chaque pression subséquente sur S1 augmente progressivement la vitesse de la chenille
- Chaque pression sur S2 résulte progressivement la vitesse de la chenille
- Une pression sur S3 arrête la chenille

Quelques points importants de notre code:

- Une fonction d'écoute de la pression sur l'un des trois boutons s'exécute à chaque boucle de décalage de la chenille; cette dernière utilise également un filtre anti rebond, empêchant un appui long d'incrémenter la vitesse plusieurs fois.
- Le facteur de vitesse est déterminé par l'élévation à la puissance v d'une valeur fixe ($0xff = 2047$). Cette opération est effectuée au moyen d'un décalage de cette constante vers la gauche de v fois.

```

1  # =====
2  # x30 : écriture = allume LEDs (bit N = LED N)
3  # x31 : lecture = état boutons (bit 0 = S0, bit 1 = S1)
4  # =====
5
6  _start:
7  addi sp, zero, 0x500    # stack base address == 0x500
8  # attends appui sur S1 (bit 1 de x31)
9  # initialiser s3 à 5
10 addi s3, zero, 5        # s3 = 5, variable pour la vitesse de la chenille
11 addi s4, zero, 0        # stop flag = 0
12
13 wait_s1:
14 andi t0, x31, 1        # isoler bit 0 (bouton S1)
15 beq  t0, zero, wait_s1  # tant que S1 n'est pas pressé, continuer d'écouter
16
17 release_s1:
18 andi t0, x31, 1
19 beq  t0, zero, done_release # on attend que le bouton S1 soit relâché
20 jal  zero, release_s1
21 done_release:
22
23 run_loop:
24 jal  ra, chenille        # démarrage de la chenille
25 beq  s4, zero, run_loop  # tant que le stop flag (s4) est à zéro, on continue
26 jal  zero, _start        # sinon, on revient au début du programme
27
28 # =====
29 # chenille : LED0→LED7 puis LED7→LED0, aller-retour
30 # =====
31
32 chenille:
33 addi sp, sp, -4
34 sw   ra, 0(sp)          # sauvegarder return address sur la pile
35
36 # Aller : LED0 → LED7
37 addi s0, zero, 1        # s0 = masque courant (LED0 = bit 0)
38 addi s1, zero, 0        # s1 = 0 -- Compteur
39 addi s2, zero, 8        # s2 = 8 (val max)
40
41 go_right:
42 add  a1, zero, s3        # s3 = 5 (2^5 = 32) pour faire 65504 cycles de delay
43 jal  ra, wait_buttons    # attendre appui sur S1 pour incrémenter delay
44
45 beq  s4, zero, not_stopped_right # si stop flag = 0, continuer la chenille
46 jal  zero, end_prog      # sinon saut vers fin du programme
47 not_stopped_right:
48
49 add  x30, zero, s0        # allumer LED courante
50 add  s3, zero, a0        # s3 = 5 (2^5 = 32) pour faire 65504 cycles de delay
51 jal  ra, delay           # attendre 65k cycles
52
53 slli s0, s0, 1          # décaler vers la gauche
54 addi s1, s1, 1          # compteur += 1
55
56 sub  t2, s2, s1          # t2 = 8 - compteur
57
58 beq  t2, zero, go_left_init # si t2 == 0 : Arrêter à LED7 (8 décalages)
59 jal  zero, go_right
60
61 go_left_init:
62 # retour : LED7 → LED0
63 # s0 est maintenant à 0x100 (trop loin), revenir à LED7

```

```

64  srli s0, s0, 1    # s0 = 0x80 = LED7
65  addi s1, zero, 0  # réinitialiser compteur
66
67  go_left:
68  add a1, zero, s3  # s3 = 5 (2^5 = 32) pour faire 65504 cycles de delay
69  jal ra, wait_buttons # attendre appui sur S1 pour incrémenter delay
70
71  beq s4, zero, not_stopped_left # si stop flag = 0, continuer la chenille
72  jal zero, end_prog # else stop
73  not_stopped_left:
74
75  add x30, zero, s0 # allumer LED courante
76  add s3, zero, a0  # s3 = 5 (2^5 = 32) pour faire 65504 cycles de delay
77  jal ra, delay     # attendre 65k cycles
78
79  srli s0, s0, 1    # décaler vers la droite
80  addi s1, s1, 1    # compteur += 1
81
82  sub t2, s2, s1    # t2 = 8 - compteur
83
84  beq t2, zero, end_prog # si t2 == 0 : Arrêter à LED0 (8 décalages)
85  jal zero, go_left
86
87  end_prog:
88  add x30, zero, zero # éteindre toutes les LEDs
89
90  lw ra, 0(sp)      # return address = sp(0)
91  addi sp, sp, 4    # réinitialiser sp
92  jalr zero, ra, 0  # retourner à ra
93
94  # =====
95  # delay : boucle d'attente (65504 cycles)
96  # =====
97
98  delay:
99  addi sp, sp, -4
100 sw ra, 0(sp) # sauvegarder return address sur la pile
101
102 # Charger 65504 dans t3
103 addi t3, zero, 0x7ff # charger 2047 sur t3
104 sll t3, t3, a0 # t3 = t3 * 2^(a0) (=65504)
105
106 delayloop:
107 # Dénombrer de 65504 à 0
108 beq t3, zero, delay_done # si t3 == 0, goto delay_done
109 addi t3, t3, -1 # t3 = t3 - 1
110
111 jal zero, delayloop # retourner au début de la boucle delayloop
112
113 delay_done:
114 lw ra, 0(sp) # return address = sp(0)
115 addi sp, sp, 4 # réinitialiser sp
116 jalr zero, ra, 0 # retourner à ra
117
118
119 # =====
120 # incrémenter/décroémenter longueur de delay
121 # =====
122 wait_buttons:
123 addi sp, sp, -4
124 sw ra, 0(sp) # sauvegarder return address sur la pile
125
126 # Stop button S3
127 andi t0, x31, 4 # isoler bit 2 (bouton S3)
128 beq t0, zero, no_stop # si S3 n'est pas pressé, continuer la chenille
129 addi s4, zero, 1 # sinon, mettre le stop flage (s4) à 1
130
131 no_stop:
132 add a0, zero, a1
133
134 andi t0, x31, 2 # isoler bit 1 (bouton S2)

```

```

135     beq  t0, zero, check_s1    # si le bouton S2 n'est pas pressé, vérifier S1
136
137     # if t0 == 1 : incrémenter delay
138     add  a0, zero, a1
139     jal  ra, incr_delay
140
141 wait_release_s2:
142     andi t0, x31, 2            # isoler bit 1 (bouton S2)
143     beq  t0, zero, done_release_s2 # si le S2 n'est plus pressé, il a donc été relâché
144     jal  zero, wait_release_s2    # si S2 est toujours pressé, attendre son relâchement
145 done_release_s2:
146     jal  zero, end_wait_buttons
147
148 check_s1:
149     andi t0, x31, 1            # isoler bit 0 (bouton S1)
150     beq  t0, zero, end_wait_buttons # si S1 n'est pas pressé, terminer la boucle de contrôle des boutons
151
152     # if t0 == 1 : décrémenter delay
153     beq  s3, zero, end_wait_buttons # si s3 == 0, ne pas décrémenter
154
155     add  a0, zero, a1
156     jal  ra, decr_delay
157
158 wait_release_s1:
159     andi t0, x31, 1
160     beq  t0, zero, done_release_s1
161     jal  zero, wait_release_s1
162 done_release_s1:
163
164 end_wait_buttons:
165     lw   ra, 0(sp)             # return address = sp(0)
166     addi sp, sp, 4             # réinitialiser sp
167     jalr zero, ra, 0           # retourner à ra
168
169 incr_delay:
170     addi a0, a0, 1             # incrémenter la vitesse de 1 : a0 = a0 + 1
171     jalr zero, ra, 0           # retourner à ra
172
173 decr_delay:
174     addi a0, a0, -1            # décrémenter la vitesse de 1 : a0 = a0 - 1
175     jalr zero, ra, 0           # retourner à ra

```

Listing 3 - Code Assembleur Chenille

11 – Conclusion

Ce projet avait pour objectif de concevoir et déployer un processeur RISC-V 32-bits réduit, l'**HEIRV32**, sur une carte FPGA, en implémentant une architecture multi-cycle.

Dans un premier temps, les spécifications ont été définies :

- **Matériel** : carte de développement FPGA EBS3 (Lattice LFE5U-25F), carte boutons/LEDs, modules PMod d'extension
- **Outils logiciels** : HDL Designer pour la conception du circuit, ModelSim pour la simulation, Lattice Diamond pour le déploiement sur FPGA
- **Fonctionnalités** : instructions minimales (R, I, lw, sw, beq, jal, jalr) et fonctions supplémentaires (gestion des boutons, code personnalisé)

La conception du processeur s'est articulée autour de deux axes. D'une part, la théorie du pipeline multi-cycle a guidé l'assemblage des blocs fournis : gestionnaire mémoire, fichier de registres, bloc d'extension, ALU et unité de contrôle. D'autre part, la **Main FSM** a été entièrement conçue et programmée pour orchestrer les signaux de contrôle à chaque étape du pipeline (Fetch → Decode → Execute → Memory Access → Write Back).

La phase de simulation ModelSim a permis de valider le comportement du circuit instruction par instruction, en corrigeant les valeurs de l'unité de contrôle au fur et à mesure. Le code assembleur a quant à lui été développé et testé via l'interpréteur RISC-V en ligne et Ripes avant intégration.

L'intégration sur FPGA a confirmé le bon fonctionnement global du système. Les points clés du bilan sont les suivants :

- **Circuit** : le processeur HEIRV32 multi-cycle répond aux spécifications minimales et s'exécute correctement sur la puce physique
- **Code assembleur** : les fonctions minimales et supplémentaires sont opérationnelles
- **Débogage** : la gestion des rebonds de boutons reste un point d'amélioration identifié ; le filtrage logiciel représente une piste de résolution directe

L'objectif principal est atteint : un processeur RISC-V réduit, capable de charger et d'exécuter un programme assembleur personnalisé depuis une carte SD, a été conçu, simulé et déployé. Ce projet illustre concrètement le lien entre les concepts théoriques de l'architecture des ordinateurs et leur réalisation physique sur un circuit programmable.

11.1 – Difficultés rencontrées

Le passage entre l'architecture single-cycle et multi-cycle a initialement posé quelques problèmes de compréhension, notamment dans la découpe des cycles en fonction des différentes opérations, ainsi que pour le calcul de PC_{next} . L'établissement de la table de vérité de l'unité de contrôle s'est également avéré être une tâche complexe.

11.2 – Etapes suivantes

Si le temps l'avait permis, un développement du support de plus d'instructions assembleur aurait été intéressant, certaines instructions s'avérant très utile pour créer des programmes plus complexes (par exemple BNE, LUI, etc).

Nous aurions pu développer d'autres programmes à exécuter sur notre processeur, tel qu'une implémentation très basique du jeu pong ;)